

Package ‘mgen’

February 28, 2019

Type Package

Title SRT Calculations

Version 0.1.0

Author Andrew, Manos and Andrei

Maintainer The package maintainer <yourself@somewhere.net>

Description More about what it does (maybe more than one line)
Use four spaces when indenting paragraphs within the Description.

License Mizuho

Encoding UTF-8

LazyData true

Depends linprog,
tidyverse,
magrittr,
markovchain,
diagram

Suggests testthat,
devtools

RoxygenNote 6.1.1

R topics documented:

.LoanAPayment	2
.loan_by_loan	3
.loop_loans	3
calculate_amortization_df	4
calculate_amortization_loan	4
calculate_cash_flow	4
calculate_simplex	5
calculate_supervisory	6
CashflowModel	6
Category	7
Category_Limits	7
CQS_from_Rating	7
Generate_Categories_List	8
Generate_Constrained_Loan_Data_Tape	8
Generate_Loan_Amounts	9

Generate_Simplex_Input_Category	9
Generate_Simplex_Input_Loan_Size	10
Generate_Simplex_Input_Obligor_Size	10
Generate_Simplex_Input_Portfolio_Size	11
Generate_Simplex_Input_WAV	11
generate_tape	11
hello	12
Initialize_Loan_Data_Tape	12
LoanAmortizationDefaultPrepayment	13
LoanPayment	13
LoanPrincipal	14
LoanPrincipalRemain	14
make_constrains	14
myrtest	15
Normalize_DataFrame	15
Perform_Obligor_Splits	15
RW_Req_SOV	16
SEC_ERBA_Choose_Points	16
SEC_ERBA_Interpolate	16
SEC_ERBA_RW	17
SEC_ERBA_Table	17
SEC_IRBA_Grularity_Type	17
SEC_IRBA_KSSFA	18
SEC_IRBA_Supervisory_Parameter	18
SEC_IRBA_Table_Parameters_Choose_Row	19
SEC_IRBA_Table_Parameters_Load	19
SEC_SA_KSSFA	20
SEC_SA_K_A	20
SEC_SA_K_SA	20
SEC_SA_RW	21
SEC_SA_Supervisory_Parameter	21
supervisoryformula	21

Index 23

.LoanAPayment	<i>Private function Loan Payment only for type A Part of an optimisation to use ifelse instead of if/else for speed purposes</i>
---------------	--

Description

Private function Loan Payment only for type A Part of an optimisation to use ifelse instead of if/else for speed purposes

Usage

```
.LoanAPayment(Term = numeric(), LoanRate = numeric(),
  OriginalBalance = numeric())
```

Arguments

Term	A number.
LoanRate	A number.
OriginalBalance	A number.

Value

A number

.loan_by_loan	<i>Goes loan by loan at the end re-adds the correct period/RW/KIRB</i>
---------------	--

Description

Goes loan by loan at the end re-adds the correct period/RW/KIRB

Usage

```
.loan_by_loan(df = data.frame())
```

Arguments

df	A Dataframe.
----	--------------

Value

A Dataframe.

.loop_loans	<i>Used as an intermediary step takes a loan list and and calculates amortization for it.</i>
-------------	---

Description

Used as an intermediary step takes a loan list and and calculates amortization for it.

Usage

```
.loop_loans(args, Term = 120)
```

Arguments

args	is a list with EAD/PEAD, RW, KIRB as Numeric elements in this order
Term	is Number

Value

A Dataframe from LoanAmortizationDefaultPrepayment

calculate_amortization_df
Amortization for a datafrmae

Description

Amortization for a datafrmae

Usage

```
calculate_amortization_df(ptf_df)
```

Arguments

ptf_df

calculate_amortization_loan
Amortization only for a loan

Description

Amortization only for a loan

Usage

```
calculate_amortization_loan(Term = numeric(), LoanRate = numeric(),
  OriginalBalance = numeric(), DefaultVector, SeverityVector,
  PrepaymentVector, RiskWeightVector, KIRBVector, LoanType)
```

Arguments

LoanType

calculate_cash_flow *Cash flow calculation Python exposed function! modify carefully and run unit tests before committing to git*

Description

Cash flow calculation Python exposed function! modify carefully and run unit tests before committing to git

Usage

```
calculate_cash_flow(theloan = data.frame(), tranches = list(),
  kirb = numeric(), elgd = numeric(), mt = numeric(), n = 500,
  supervisory_type = character(), pool_type = character())
```

Arguments

theloan	A Dataframe
tranches	A List
kirb	A Number
mt	A Number
n	A Number (default 500)
supervisory_type	A String
pool_type	A String
elgdA	A Number

Value

A Dataframe

calculate_simplex	<i>Simplex calculation Python exposed!</i>
-------------------	--

Description

Simplex calculation Python exposed!

Usage

```
calculate_simplex(tape_df, target = numeric(),
                  criterias = list(character(), character(), numeric()))
```

Arguments

tape_df	A Dataframe
target	A Number
criterias	A List of lists

Value

A Dataframe

`calculate_supervisory` *Python exposed New IRBA Supervisory calculation*

Description

Python exposed New IRBA Supervisory calculation

Usage

```
calculate_supervisory(Pool_Type = character(), N = numeric(),
  Rank = character(), K_IRB = numeric(), LGD = numeric(),
  Mt = numeric(), Attachment_Point = numeric(),
  Detachment_Point = numeric(), RW_Floor = 0.15)
```

Arguments

Pool_Type	A String (Wholesale or Retail)
N	A Number
Rank	A Number
K_IRB	A Number
LGD	A Number
Mt	A Number
Attachment_Point	A Number
Detachment_Point	A Number
RW_Floor	A Number (Default 0.15!)

Value

A Number

CashflowModel	<i>Title</i>
---------------	--------------

Description

Title

Usage

```
CashflowModel(theloan = list(), tranches = list(), kirb = numeric(),
  elgd = numeric(), mt = numeric(), n = 500,
  supervisory_type = character(), pool_type = character())
```

Arguments

pool_type

Category	Create a dataframe for a categorical variable The minimum input is a string with labels for the category
----------	--

Description

Create a dataframe for a categorical variable The minimum input is a string with labels for the category

Usage

```
Category(Labels = character(), Weights = 1, Number = 10)
```

Arguments

Number

Category_Limits	Creates the limit structure input for any categorical constraint
-----------------	--

Description

Creates the limit structure input for any categorical constraint

Usage

```
Category_Limits(Labels = character(), Limits = numeric())
```

Arguments

Limits

CQS_from_Rating	<i>CQS_from_Rating</i>
-----------------	------------------------

Description

CQS_from_Rating

Usage

```
CQS_from_Rating(Rating, Rating_Style = "S_and_P")
```

Arguments

Rating_Style

Generate_Categories_List

Automatically create a list of dataframes for the categories by passing a generic name for each category as part of an input string

Description

Automatically create a list of dataframes for the categories by passing a generic name for each category as part of an input string

Usage

```
Generate_Categories_List(Categories_Names = character())
```

Arguments

Categories_Names
A String

Generate_Constrained_Loan_Data_Tape

Generates a sufficiently large loan amount vector Applies the relevant label for every category according to the passed distributions The data tape created here is on an obligor basis

Description

Generates a sufficiently large loan amount vector Applies the relevant label for every category according to the passed distributions The data tape created here is on an obligor basis

Usage

```
Generate_Constrained_Loan_Data_Tape(Data_Tape_EAD_Size_Target = numeric(),  
Data_Tape_Loan_Size_Mean = numeric(),  
Data_Tape_Loan_Size_Dev = numeric(), Categories_List = list())
```

Arguments

Data_Tape_EAD_Size_Target
A Number
Data_Tape_Loan_Size_Mean
A Number
Data_Tape_Loan_Size_Dev
A Number
Categories_List
A List

Generate_Loan_Amounts	<i>Generate a vector of loan amounts drawn from the normal distribution with particular mean and variance. The vector contains a number of portfolios (default is 2) of target portfolio size The structure of this generation is such in order to ensure that when selecting from this portfolio according to constraints you always have sufficient loan stock</i>
-----------------------	--

Description

Generate a vector of loan amounts drawn from the normal distribution with particular mean and variance. The vector contains a number of portfolios (default is 2) of target portfolio size The structure of this generation is such in order to ensure that when selecting from this portfolio according to constraints you always have sufficient loan stock

Usage

```
Generate_Loan_Amounts(Portfolio_Size = numeric(),
  Loan_Mean = numeric(), Loan_Dev = numeric(), Portfolios = 2)
```

Arguments

Portfolio_Size	A Number
Loan_Mean	A Number
Loan_Dev	A Number
Portfolios	A Number (Default 2)

Generate_Simplex_Input_Category	<i>Creates the limit structure input for any categorical constraint</i>
---------------------------------	---

Description

Creates the limit structure input for any categorical constraint

Usage

```
Generate_Simplex_Input_Category(Limits = list(), Column = character())
```

Arguments

Column

Generate_Simplex_Input_Loan_Size

Creates the constraints that you can not select more than the available size of each loan

Description

Creates the constraints that you can not select more than the available size of each loan

Usage

```
Generate_Simplex_Input_Loan_Size(Loan_Amounts = numeric())
```

Arguments

Loan_Amounts

Generate_Simplex_Input_Obligor_Size

Creates the constraints for obligor concentrations where all obligors have the same common concentration limit

Description

Creates the constraints for obligor concentrations where all obligors have the same common concentration limit

Usage

```
Generate_Simplex_Input_Obligor_Size(Obligor_Limit_Common = numeric(),  
  Obligor_IDs)
```

Arguments

Obligor_IDs

Generate_Simplex_Input_Portfolio_Size

Creates the constraint of the total portfolio size selected is less than a target size

Description

Creates the constraint of the total portfolio size selected is less than a target size

Usage

```
Generate_Simplex_Input_Portfolio_Size(Loan_Amounts = numeric(),
Portfolio_Size = numeric())
```

Arguments

Portfolio_Size

Generate_Simplex_Input_WAV

WAV Constraint

Description

WAV Constraint

Usage

```
Generate_Simplex_Input_WAV(Limit, Column = numeric())
```

Arguments

Column

generate_tape

Generator function exposed to python!

Description

Generator function exposed to python!

Usage

```
generate_tape(categories = list(character(), character(), numeric()),
est = 1e+08, lsm = 50000, lsd = 12500, lgd = 0.4,
pd_map = c(0.00324916801, 0.01765866569, 0.1983357249, 0.4616194311,
2.109451063, 7.848052027, 20.49498387, 100), mmin = 0.01, mmax = 10)
```

Arguments

categories	A List
est	A Number (default 10e7)
lsm	A Number (default 50e3)
lsd	A Number (default 12.5e3)
lgd	A Number (default 0.4)
pd_map	A List (default c(0.00324916801, 0.01765866569, 0.19833572490, 0.46161943110, 2.10945106300, 7.84805202700, 20.49498387000, 100.00000000000))
mmin	A Number (default 0.01)
mmax	A Number (default 10)

Value

A Dataframe

hello	<i>Hello, World!</i>
-------	----------------------

Description

Prints 'Hello, world!'.

Usage

```
hello()
```

Examples

```
hello()
```

Initialize_Loan_Data_Tape	<i>Initialises a data tape dataframe for a particular number of rows and specific column names</i>
---------------------------	--

Description

Initialises a data tape dataframe for a particular number of rows and specific column names

Usage

```
Initialize_Loan_Data_Tape(Rows_Number = integer(),
  Column_Names = list())
```

Arguments

Rows_Number	A Number (int)
Column_Names	A List

LoanAmortizationDefaultPrepayment
Amoritization Prepayment Function

Description

Amoritization Prepayment Function

Usage

```
LoanAmortizationDefaultPrepayment(Term = numeric(),
  LoanRate = numeric(), OriginalBalance = numeric(), DefaultVector,
  SeverityVector, PrepaymentVector, RiskWeightVector, KIRBVector, LoanType)
```

Arguments

Term	A Number
LoanRate	A Number
OriginalBalance	A Number
DefaultVector	A Vector
SeverityVector	A Vector
PrepaymentVector	A Vector
RiskWeightVector	A Vector
KIRBVector	A Vector
LoanType	A string (A or B)

Value

A Dataframe

LoanPayment	<i>Loan payment</i>
-------------	---------------------

Description

Loan payment

Usage

```
LoanPayment(Term = numeric(), LoanRate = numeric(),
  OriginalBalance = numeric(), LoanType = character())
```

Arguments

LoanType

LoanPrincipal	<i>Loan Principal</i>
---------------	-----------------------

Description

Loan Principal

Usage

```
LoanPrincipal(Term = numeric(), LoanRate = numeric(),
  OriginalBalance = numeric(), Period = numeric(), LoanType)
```

Arguments

LoanType

LoanPrincipalRemain	<i>Loan principal remaian</i>
---------------------	-------------------------------

Description

Loan principal remaian

Usage

```
LoanPrincipalRemain(Term = numeric(), LoanRate = numeric(),
  OriginalBalance = numeric(), Period = numeric(), LoanType)
```

Arguments

LoanType

make_constrains	<i>Transforms criterias from python input to R (Private function)</i>
-----------------	---

Description

Transforms criterias from python input to R (Private function)

Usage

```
make_constrains(tape_df, target = numeric(),
  criterias = list(character(), character(), numeric()))
```

Arguments

criterias

myrtest	<i>Stat Test? To document further and decided if exposed publicly (currently not)</i>
---------	---

Description

Stat Test? To document further and decided if exposed publicly (currently not)

Usage

```
myrtest(count_sims = 1)
```

Arguments

count_sims

Normalize_DataFrame	<i>Normalize percents and commas separated values from a given Dataframe removes commas separator from numbers ex: (100,000 -> 1000000) and converts percents into numeric (10 Normalize_DataFrame(df = data.frame()) \itemdfA Dataframe A Dataframe Normalize percents and commas separated values from a given Dataframe removes commas separator from numbers ex: (100,000 -> 1000000) and converts percents into numeric (10</i>
---------------------	--

Perform_Obligor_Splits	<i>Takes an "obligor basis" data tape and splits the amount for each obligor in a random number of loans in random amounts</i>
------------------------	--

Description

Takes an "obligor basis" data tape and splits the amount for each obligor in a random number of loans in random amounts

Usage

```
Perform_Obligor_Splits(Data_Tape = data.frame(),
  Obligor_Min = integer(), Obligor_Max = integer())
```

Arguments

Data_Tape	A Dataframe
Obligor_Min	A Number (integer only)
Obligor_Max	A Number (integer only)

RW_Req_SOV	<i>The formula that calculates the risk weight percentage for sovereigns and large corporate exposures</i>
------------	--

Description

The formula that calculates the risk weight percentage for sovereigns and large corporate exposures

Usage

```
RW_Req_SOV(PD = numeric(), LGD = numeric(), M = numeric())
```

Arguments

PD	A Number
LGD	A Number
M	A Number

SEC_ERBA_Choose_Points

Returns the two RW pillar points to interpolate from

Description

Returns the two RW pillar points to interpolate from

Usage

```
SEC_ERBA_Choose_Points(CQS = numeric(), STS = logical(),  
  Senior = logical())
```

Arguments

Senior

SEC_ERBA_Interpolate *Returns the interpolated RW*

Description

Returns the interpolated RW

Usage

```
SEC_ERBA_Interpolate(Maturity = numeric(), CQS = numeric(),  
  STS = logical(), Senior = logical())
```

Arguments

Senior

SEC_ERBA_RW	<i>RW adjusted for tranche thickness</i>
-------------	--

Description

RW adjusted for tranche thickness

Usage

```
SEC_ERBA_RW(Attachment_Point = numeric(), Detachment_Point = numeric(),  
            Maturity = numeric(), CQS = numeric(), STS = logical(),  
            Senior = logical())
```

Arguments

Senior

SEC_ERBA_Table	<i>Loads the SEC-ERBA Risk Weight tables</i>
----------------	--

Description

Loads the SEC-ERBA Risk Weight tables

Usage

```
SEC_ERBA_Table()
```

SEC_IRBA_Granularity_Type	<i>SEC_IRBA_Granularity_Type</i>
---------------------------	----------------------------------

Description

SEC_IRBA_Granularity_Type

Usage

```
SEC_IRBA_Granularity_Type(N = numeric())
```

Arguments

N A Number

SEC_IRBA_KSSFA	<i>Calculates the KSSFA term which is is the capital requirement per unit of securitisation exposure under the SEC-IRBA</i>
----------------	---

Description

Calculates the KSSFA term which is is the capital requirement per unit of securitisation exposure under the SEC-IRBA

Usage

```
SEC_IRBA_KSSFA(Pool_Type = character(), N = numeric(),
  Rank = character(), K_IRB = numeric(), LGD = numeric(),
  Mt = numeric(), Attachment_Point, Detachment_Point = numeric())
```

Arguments

Pool_Type	A String (Wholesale or Retail)
N	A Number
Rank	A String (Senior or Non-senior)
K_IRB	A Number
LGD	A Number
Mt	A Number
Attachment_Point	
	A Number
Detachment_Point	
	A Number

Value

A Number

SEC_IRBA_Supervisory_Parameter	<i>Calculates the supervisory parameter p which is an intermediate variable for the SEC_IRBA RW</i>
--------------------------------	---

Description

Calculates the supervisory parameter p which is an intermediate variable for the SEC_IRBA RW

Usage

```
SEC_IRBA_Supervisory_Parameter(Pool_Type = character(), N = numeric(),
  Rank = character(), K_IRB = numeric(), LGD = numeric(),
  Mt = numeric())
```

Arguments

Pool_Type	A String (Wholesale or Retail)
N	A Number
Rank	A String (Senior or Non-senior)
K_IRB	A Number
LGD	A Number
Mt	A Number

SEC_IRBA_Table_Parameters_Choose_Row

Returns the relevant row of table with the onstants needed for the supervisory parameter p (BIS 374 doc)

Description

The row decision is based on:

Usage

```
SEC_IRBA_Table_Parameters_Choose_Row(Pool_Type = character(),
  N = numeric(), Rank = character())
```

Arguments

Pool_Type	A String (Wholesale or Retail)
N	A Number
Rank	A String (Senior or Non-senior)

SEC_IRBA_Table_Parameters_Load

SEC_IRBA_Table_Parameters_Load

Description

SEC_IRBA_Table_Parameters_Load

Usage

```
SEC_IRBA_Table_Parameters_Load()
```

SEC_SA_KSSFA	<i>Calculates the KSSFA term for SEC-SA</i>
--------------	---

Description

Calculates the KSSFA term for SEC-SA

Usage

```
SEC_SA_KSSFA(Attachment_Point = numeric(),  
             Detachment_Point = numeric(), RW = numeric(), W = numeric(),  
             W_Known = numeric(), STS = logical())
```

Arguments

STS

SEC_SA_K_A	<i>Calculates the K_A term for SEC-SA</i>
------------	---

Description

Calculates the K_A term for SEC-SA

Usage

```
SEC_SA_K_A(RW = numeric(), W = numeric(), W_Known = numeric())
```

Arguments

W_Known

SEC_SA_K_SA	<i>Returns the K_SA value</i>
-------------	-------------------------------

Description

Returns the K_SA value

Usage

```
SEC_SA_K_SA(RW = numeric())
```

Arguments

RW

SEC_SA_RW	<i>Calculates the SEC-SA RW</i>
-----------	---------------------------------

Description

Calculates the SEC-SA RW

Usage

```
SEC_SA_RW(Attachment_Point = numeric(), Detachment_Point = numeric(),  
          RW = numeric(), W = numeric(), W_Known = numeric(),  
          STS = logical(), RW_Floor = 0.15)
```

Arguments

RW_Floor

SEC_SA_Supervisory_Parameter	<i>Returns the supervisory parameter p in the SEC-SA context</i>
------------------------------	--

Description

Returns the supervisory parameter p in the SEC-SA context

Usage

```
SEC_SA_Supervisory_Parameter(STS = logical())
```

Arguments

STS

supervisoryformula	<i>Old supervisory formula</i>
--------------------	--------------------------------

Description

Old supervisory formula

Usage

```
supervisoryformula(L, T, KIRB, ELGD, N)
```

Arguments

L	A Number
T	A Number
KIRB	A Number
ELGD	A Number
N	A Number

Value

A number

Index

.LoanAPayment, [2](#)
.loan_by_loan, [3](#)
.loop_loans, [3](#)

calculate_amortization_df, [4](#)
calculate_amortization_loan, [4](#)
calculate_cash_flow, [4](#)
calculate_simplex, [5](#)
calculate_supervisory, [6](#)
CashflowModel, [6](#)
Category, [7](#)
Category_Limits, [7](#)
CQS_from_Rating, [7](#)

Generate_Categories_List, [8](#)
Generate_Constrained_Loan_Data_Tape, [8](#)
Generate_Loan_Amounts, [9](#)
Generate_Simplex_Input_Category, [9](#)
Generate_Simplex_Input_Loan_Size, [10](#)
Generate_Simplex_Input_Obligor_Size, [10](#)
Generate_Simplex_Input_Portfolio_Size, [11](#)
Generate_Simplex_Input_WAV, [11](#)
generate_tape, [11](#)

hello, [12](#)

Initialize_Loan_Data_Tape, [12](#)

LoanAmortizationDefaultPrepayment, [13](#)
LoanPayment, [13](#)
LoanPrincipal, [14](#)
LoanPrincipalRemain, [14](#)

make_constrains, [14](#)
myrtest, [15](#)

Normalize_DataFrame, [15](#)

Perform_Obligor_Splits, [15](#)

RW_Req_SOV, [16](#)

SEC_ERBA_Choose_Points, [16](#)
SEC_ERBA_Interpolate, [16](#)
SEC_ERBA_RW, [17](#)
SEC_ERBA_Table, [17](#)
SEC_IRBA_Granularity_Type, [17](#)
SEC_IRBA_KSSFA, [18](#)
SEC_IRBA_Supervisory_Parameter, [18](#)
SEC_IRBA_Table_Parameters_Choose_Row, [19](#)
SEC_IRBA_Table_Parameters_Load, [19](#)
SEC_SA_K_A, [20](#)
SEC_SA_K_SA, [20](#)
SEC_SA_KSSFA, [20](#)
SEC_SA_RW, [21](#)
SEC_SA_Supervisory_Parameter, [21](#)
supervisoryformula, [21](#)